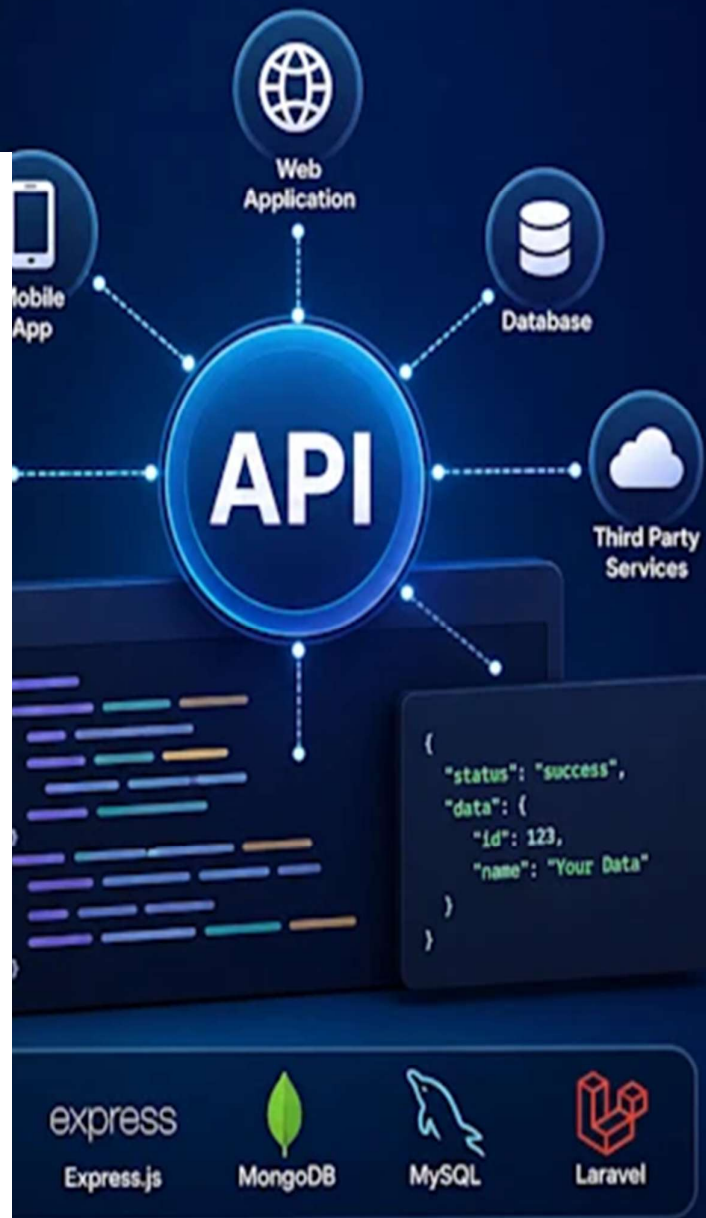


JavaScript-10

Webb-API:er

26 september 2026

JavaScript – utan tjafs
Författare: Acke Strömberg



För en kanin

Webb-API:er

Webbläsaren innehåller mycket mer än bara kod för att visa HTML och CSS. Den har en mängd inbyggda verktyg och superkrafter som kallas för Web-API:er. I det här kapitlet ska vi titta närmare på två av de absolut mest användbara verktygen: hur vi sparar data direkt på användarens dator så att den finns kvar när sidan laddas om, samt hur vi skapar smart **infinite scroll** och **lazy-loading** med ett modernt verktyg som heter **Intersection Observer**.

Web Storage – Kom ihåg data i webbläsaren

När en användare laddar om en webbsida, eller navigerar till en ny undersida, nollställs alla JavaScript-variabler i minnet. Om du vill spara produkter i en kundvagn, eller komma ihåg om användaren har valt "mörkt tema", måste datan sparas någonstans.

För detta har vi två fantastiska verktyg:

- **LocalStorage:** Sparar datan permanent. Den finns kvar även om användaren stänger ner webbläsaren eller startar om datorn.
- **SessionStorage:** Sparar datan temporärt. Datan raderas automatiskt så fort användaren stänger den specifika fliken.

Viktigt att veta: Båda dessa system kan bara spara **textsträngar**. Om vi vill spara objekt eller arrayer måste vi omvandla dem till JSON-strängar först!

```
// Data vi vill spara
const installningar = {
  tema: 'morkt',
  volym: 80
};

// 1. SPARA: Vi gör om objektet till text med JSON.stringify() innan vi sparar
localStorage.setItem('anvandar_val', JSON.stringify(installningar));

// ... Sidan laddas om eller användaren stänger fliken ...

// 2. HÄMTA: Vi hämtar textsträngen från minnet
const sparadText = localStorage.getItem('anvandar_val');

// 3. OMVANDLA: Vi gör om texten till ett JavaScript-objekt igen med JSON.parse()
const laddadeInstallningar = JSON.parse(sparadText);

console.log(laddadeInstallningar.tema); // Utskrift: morkt
```

Bild 1 Med JSON.stringify och JSON.parse kan vi enkelt spara och hämta komplex data.

Intersection Observer – Styr vad som syns på skärmen

Förr i tiden, om man ville veta när ett visst element dök upp på skärmen (t.ex. för att starta en animation eller ladda fler produkter när användaren scrollat till botten), var man tvungen att lyssna på webbläsarens scroll-event. Det gjorde sidan väldigt långsam eftersom scroll-funktionen triggades hundratals gånger i sekunden.

Idag använder vi **Intersection Observer API**. Det är som att sätta en dold kamera på ett specifikt HTML-element på sidan. Vi ber webbläsaren att hålla utkik och säga till *bara* när elementet faktiskt kliver in i eller lämnar användarens synfält (skärmen).

Detta är det perfekta verktyget för att bygga **Infinite Scroll** (att ladda mer innehåll automatiskt när man når botten) eller **Lazy Loading** (att inte ladda tunga bilder förrän användaren faktiskt scrollat ner till dem).

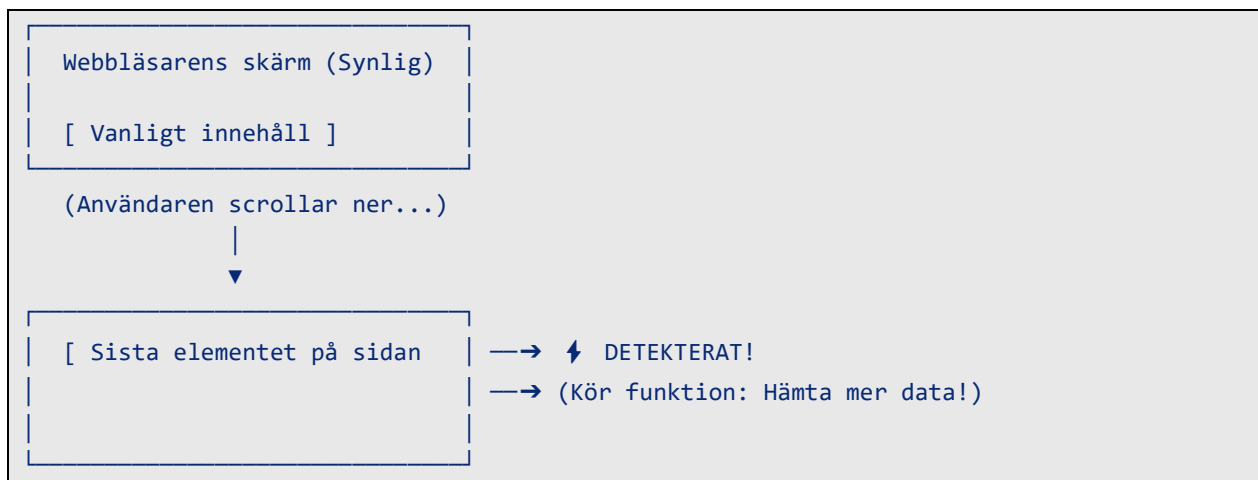


Bild 2 Observatören utlöser vår funktion när övervakade elementet blir synligt för användaren.

Här lägger vi en osynlig "laddnings-låda" längst ner på vår sida. När den syns på skärmen vet vi att det är dags att hämta mer data.

```
// 1. Funktionen som ska köras när elementet syns på skärmen
const hanteraSynlighet = (entries) => {
  entries.forEach((entry) => {
    // entry.isIntersecting blir TRUE i det ögonblick elementet syns på skärmen
    if (entry.isIntersecting) {
      console.log("Användaren har nått botten! Hämtar fler produkter...");
      // Här skulle du göra ett fetch-anrop för att hämta mer data!
    }
  });
};

// 2. Skapa själva observatören (kameran)
const observator = new IntersectionObserver(hanteraSynlighet);

// 3. Säg åt kameran vilket element den ska hålla koll på
const laddningsMarkor = document.querySelector('#bottom-marker');
observator.observe(laddningsMarkor);
```

Bild 3 Genom att använda IntersectionObserver reagerar koden bara när det behövs.